

LSTM Sequence-to-Sequence vs. a Pretrained LLM on Abstractive Summarization

System report. 5 pages excluding references and appendices.

1. Task and dataset

Abstractive single-document summarization on **CNN/DailyMail 3.0.0** (`abisee/cnn_dailymail`, **Apache-2.0**; Hermann et al., 2015; See et al., 2017). The official splits are article-disjoint; we use them unchanged, adding only seeded subsampling for tractability: **79,996** train, **3,000** validation, **11,490** test (mean article 785 tokens, mean summary 55).

Every system is scored on the same 500-article subset of test, drawn once with seed 1234 before any model development, with indices recorded in `dataset_meta.json`.

We verified leakage rather than assuming it: article-ID overlap between splits is exactly zero, the 500-article set is a subset of test disjoint from train, and the four few-shot exemplars sent to the LLM come from train.

Pipeline validation. Before comparing anything we reproduced a published baseline. Our Lead-3 scores **40.04 / 17.50 / 36.34** (ROUGE-1/2/Lsum) on the full test set, against See et al.'s published **40.34 / 17.70 / 36.57**. Agreement to about 0.3 ROUGE indicates the tokenization, sentence splitting, reference construction and ROUGE configuration are correct. We report Lead-3 throughout, since on this dataset it is a famously strong baseline and a summarization score is hard to interpret without it.

2. System design

2.1 LSTM seq2seq with attention

Implemented directly in PyTorch from `nn.LSTM`, `nn.Embedding`, `nn.Linear` and `nn.Dropout` only. No prebuilt seq2seq framework appears anywhere in the model, the training loop or decoding.

```
Embedding(50,000 x 256, shared) -> BiLSTM encoder (256/direction)
-> bridge tanh(W.[h_fwd ; h_bwd]) -> Bahdanau attention, masked over padding
-> LSTM decoder (256, input feeding) -> tanh(W_c.[h_t ; c_t])
-> output projection (weight-tied to the embedding)
```

The model has **15,347,280 parameters**, of which 12.8M (83%) is the embedding table, so the sequence model itself is about 2.5M.

Preprocessing. Word-level regex tokenizer, lowercased, with abbreviation-aware sentence splitting. We chose word-level over subwords deliberately, because it makes the OOV failure mode observable, which a subword vocabulary hides by construction. The vocabulary is 50,000 types from train only, covering **98.17%** of training tokens. Sources are truncated to 400 tokens (following See et al.) and targets to 100.

Training. Teacher forcing, label-smoothed cross-entropy (0.1), Adam at 1e-3, gradient clipping 5.0, `ReduceLRonPlateau`, early stopping patience 2, 5 epochs, batch 64 with length-bucketed batching.

Decoding. Beam 4 with the GNMT length penalty and repeated-trigram blocking, with `<pad>` and `<unk>` suppressed. Greedy and no-blocking variants are reported as ablations.

Three implementation details mattered, each found by measuring rather than by reading code. Padding is masked before the attention softmax; without that, 63% of attention mass lands on padding for a short article in a mixed batch. The vocabulary projection is applied in time-chunks instead of as one 1.28 GB (`B, T, V`) tensor. Padded shapes are quantized so the Metal backend stops recompiling a kernel per distinct tensor shape, a **34-55x speedup** that made training feasible. Full accounts are in **Appendix E**.

2.2 LLM baseline

Llama 3.1 8B Instruct, 4-bit, run **locally** on the Apple silicon GPU via MLX, the assignment's free open-weights option, so we report cost as GPU-hours. Decoding is greedy, so the baseline is deterministic. This measures the gap

to a mid-size open model, a lower bound on the gap to a frontier model. The harness also supports a hosted API backend unchanged.

Two prompt variants, differing on whether the prompt describes the reference style. **A ("plain")** is a natural request; **B ("style-matched")** adds the length (about 55 words), sentence count (3 to 4) and register of CNN/DailyMail highlights. Each runs zero-shot and few-shot ($k = 4$), giving four settings. Exact prompts in **Appendix A**.

Input parity. The LLM receives the article truncated to the same 400-word window the LSTM encoder sees; without parity we would be comparing amounts of input rather than models. The window discards **49% of all article tokens** and truncates **82% of articles**, equally for both. We also ran the unmatched condition to check this did not handicap the LLM, and it did not (§6.3).

3. Experimental settings

Apple M4 Pro (16-core GPU), 24 GB unified memory, macOS 15; PyTorch 2.13 (MPS), Python 3.13. Seed 1234 everywhere. Metric: ROUGE-1/2/Lsum F1 (`rouge-score, use_stemmer=True`), 95% percentile bootstrap CIs over 1,000 resamples. Differences between systems use a **paired bootstrap** (10,000 replicates, resampling articles once per replicate and applying the same resample to both), because systems scored on the same articles are correlated and independent intervals are too conservative.

4. Results

Full tables in **Appendix F**; per-epoch curves in `runs/*/train_summary.json`.

System	ROUGE-1	ROUGE-2	ROUGE-Lsum	Len
LLM B (style-matched), zero-shot	41.35 [40.44, 42.20]	18.07	38.14	82
LLM B, zero-shot, full article	40.87 [39.95, 41.70]	17.70	37.65	84
LLM B, few-shot $k=4$	40.48 [39.62, 41.34]	16.58	37.58	79
Lead-3 baseline	39.89 [38.87, 40.93]	17.60	36.35	86
LLM A (plain), few-shot $k=4$	39.40 [38.60, 40.21]	14.90	36.15	81
LLM A, zero-shot	38.52 [37.63, 39.34]	14.91	34.65	110
LSTM + attention (beam 4)	35.00 [34.03, 36.04]	13.75	32.25	48

4.1 The gap, and what it is made of

Paired bootstrap against the LSTM: the best LLM setting is **+6.35 ROUGE-1** [+5.34, +7.39], $p < 0.0001$, winning on 358 of 500 articles.

The two prompts differ by 2.83 ROUGE-1 (41.35 against 38.52 zero-shot), which is **45% of the model gap from phrasing alone**. Reporting only variant A would have shown a 3.5-point gap; only variant B, 6.4. Both are defensible single numbers, and both misattribute specification to capability. This is the strongest argument for testing more than one prompt.

Lead-3 beats four of the five LLM configurations, and on ROUGE-2 it loses only to variant B zero-shot, by 0.47. A rule with no parameters outscores an 8B pretrained model unless that model is told what the references look like.

Few-shot helps the weak prompt (+0.88) and hurts the strong one (-0.87). Exemplars and an explicit specification are substitutes: once the target has been stated, four examples add variance without adding information. Variant B's novel-bigram rate rises from 0.246 to 0.370 with exemplars, so it drifts away from the style it was told to adopt.

4.2 Consistency across length and difficulty

The gap is broadly constant and does not widen with article length (+7.18 short, +5.13 medium, +6.74 long). This contradicts the obvious hypothesis: if the LSTM's limit were the recurrent bottleneck compressing long inputs, the gap should grow. It does not. The buckets cannot really test it either, since 82% of articles already exceed the encoder window and differ only in how much was discarded.

All systems degrade sharply on abstractive examples (LSTM -9.5 ROUGE-1, LLM -10.6, Lead-3 -10.7) and degrade together, which says more about ROUGE than about the models.

4.3 Ablations

Identical hyperparameters, data and seed; exactly one config line differs in each. Paired bootstrap against the full model.

Variant	Val PPL	ROUGE-1	Δ	p
LSTM + attention (beam 4)	35.6	35.00		
100-token encoder window	65.5	34.02	-0.98	0.053 (<i>n.s.</i>)
unidirectional encoder	40.0	33.12	-1.87	0.0001
greedy decoding	35.6	32.60	-2.40	<0.0001
no trigram blocking	35.6	29.65	-5.35	<0.0001
no attention	121.7	20.97	-14.02	<0.0001

Attention is doing most of the work. Removing it costs 14.02 ROUGE-1 and collapses ROUGE-2 fourfold (13.75 to 3.47). Unigram overlap at 21 with near-zero bigram overlap is the signature of summary-shaped text not about this article: 53% of its content words are absent from the source, against 1.5% for the full model. This is the fixed-vector bottleneck; without attention the decoder cannot select which part of a 400-token article to describe.

Repetition costs 5.35 ROUGE-1 and is a decoding artifact. Without trigram blocking the duplicate-trigram rate is 0.282; with it, exactly 0.0. The canonical LSTM failure is real but correctable at decoding time, not intrinsic to the trained model.

A 4x smaller encoder window costs nothing measurable ($p = 0.053$, ROUGE-2 $p = 0.495$, win/loss 235/265) despite validation perplexity nearly doubling. The model is measurably worse at modelling the text yet produces summaries of indistinguishable quality. Since the window already discards 49% of tokens, we read this as evidence of lead bias, and as part of why Lead-3 stays competitive.

4.4 Cost, latency, and compute

Measured on the same machine (full table in Appendix F). The LSTM is **15.3M parameters / 61 MB** against about 8B / 4.5 GB, cost **3.05 GPU-hours** to train, and generates a summary in **0.231 s** (259/min). The LLM needs **2.85 s** zero-shot (21/min) and 9.2 s few-shot (6.5/min), so the LSTM is **12x faster per summary** than the fastest LLM setting, against a quantized 8B model on the same GPU with no network hop. Total LLM generation was **4.07 GPU-hours for 2,500 summaries** across five settings; both systems cost \$0.00, running locally. Few-shot triples generation cost (2,200 extra prefill tokens per request) for no benefit on the better prompt.

5. Error analysis

Thirteen examples selected by behaviour rather than score are in `results/qualitative.md`. We state each textbook failure mode below only where the data supports it; one is refuted.

Rare-word breakage (LSTM), confirmed but invisible in the obvious metric. The measured OOV rate is **0.000** for every LSTM system, an artifact: `<unk>` is suppressed, so the model cannot emit an OOV token (Lead-3, copying real text, shows the true rate of 0.023). The failure relocates to substitution and omission, and **67% of references contain at least one token the model cannot produce**: `fredric brandt` becomes `dr. frederic brandt`, an in-vocabulary misspelling of a real person that leaves no metric signature. Evidence in Appendix G.

Fluent-but-wrong, confirmed, and not confined to the ablation. The no-attention model, given a Louisville fire, generated "the fire is the fire in the city of san diego." The full model is far better (1.5% unsupported content) but not immune: two of thirteen qualitative examples contain invented content, one leaving **no lexical signature at all** (Appendix G.5).

"The LLM hallucinates", refuted. Unsupported-content is a lexical proxy: on constructed cases a faithful paraphrase scored 0.857 while a fabrication reusing source words scored 0.300 (Appendix G.2). Variant B's 0.055 alongside a 0.246 novel-bigram rate is restrained paraphrase, not invention. A real hallucination rate would need human annotation, which we did not perform.

Format drift (LLM), confirmed, and its largest weakness. Variant A zero-shot produces **110-token** summaries against a 58-token reference. Variant B, which specifies 3 to 4 sentences and roughly 55 words, produces 82 and gains 2.83 ROUGE-1. Its headline failure is not faithfulness but failing to infer an output specification that was never stated.

The two fail in opposite directions. The LSTM's errors are omission: it copies (novel-bigram 0.080), stays short (48 tokens against 58), drops what it cannot say. The LLM's are excess: it rewrites (0.246), over-produces (82), adds correct but unrequested detail. In one example the LSTM beats the LLM 52.6 to 42.0 because the LLM added true information the reference omitted; ROUGE rewards the LSTM's direction more than its quality warrants.

6. Discussion

6.1 Why the gap exists

Pretraining and transfer dominate. The LSTM sees 80k pairs and nothing else, so it must learn English, news register and the summarization objective at once, with 83% of its parameters spent on an embedding table. That the LLM reaches 41.35 with no gradient step on this data is the transfer-learning result. Capacity matters less than the parameter counts suggest: a 500x difference yields an 18% relative ROUGE-1 difference.

The recurrent bottleneck binds only when attention is absent. Removing attention costs 14.02 ROUGE-1, and self-attention solves the same problem better. But a 4x smaller window costs nothing significant and the gap does not widen with length, so on this dataset the LSTM fails at knowing what a sentence means rather than at carrying information across 400 timesteps. Specification accounts for a further 45% of the gap (§4.1), which no architectural account explains.

6.2 Failure-mode contrast

Novel-bigram rate is 0.080 for the LSTM against 0.246 for the LLM, unsupported content 0.015 against 0.055, length 48 against 82 tokens for a 58-token reference. 92% of the LSTM's bigrams appear verbatim in the source, putting it closer to a learned extractive system than an abstractive one, while the LLM produces 41% more text and rewrites a quarter of it. ROUGE is not neutral between these behaviours, since it rewards copying and penalises paraphrase. That the LLM wins anyway suggests the true gap is larger than 6.35 points, while Lead-3, pure copying, outscores most LLM settings.

6.3 Fairness of the comparison

The comparison is unfair to the LSTM: the LLM was pretrained on a corpus larger by many orders of magnitude, and CNN/DailyMail is among the most mirrored NLP benchmarks, so its zero-shot performance may partly reflect memorization.

It is also unfair to the LLM. The LSTM is trained on this dataset's reference distribution and so optimizes the exact stylistic target ROUGE rewards, while the LLM must be told through a prompt. The A-against-B gap of **2.83 ROUGE-1** is the penalty, 45% of the total gap.

Input parity does not handicap the LLM, and we measured that rather than assuming it. Given the untruncated article, variant B zero-shot scores **40.87** against **41.35** on the matched window, a difference of **-0.47 ROUGE-1** [-0.91, -0.03], $p = 0.034$. More input made it slightly worse, consistent with §4.3 and with Lead-3's strength, since the summary-relevant content sits in the opening. The parity choice costs the LLM nothing.

A fairer middle point would be fine-tuning a small pretrained transformer (BART-base, T5-small) on the same 80k pairs, isolating pretraining from architecture. Our results predict it would close most of the gap.

6.4 Engineering trade-offs

Latency: 0.231 s against 2.85 s, a factor of 12, against a quantized 8B model on the same GPU with no network hop, so any sub-second budget excludes the LLM before cost is considered. **Throughput:** 259 against 21 summaries/min, which at a million documents is about 2.7 GPU-days against 33, with the 3.05-hour training cost repaid after roughly 4,200 documents. **Deployment:** 61 MB against 4.5 GB, allowing a commodity CPU, embedded hardware or an air-gapped network. **Controllability:** our failure modes are bounded and fixable at decoding (repetition eliminated, length governed by the GNMT penalty, `<unk>` suppressed), whereas the LLM ignored an explicit length instruction by 41% and the remedy there is prompt iteration with no guarantee. **Low-resource settings:** the LLM's advantage comes from pretraining, so where that pretraining does not exist it disappears, while 80k supervised pairs remains achievable.

Where the LLM clearly wins is any task with no training data. It scored 41.35 having never seen this dataset, and we have no counterpart to that.

6.5 Limitations and ethics

The models are undertrained, and we can bound by how much. All four runs hit the 5-epoch cap with validation loss still improving; early stopping never fired. The per-epoch perplexity gain halved consistently (335.8, 70.0, 47.0, 39.4, 35.6), extrapolating to about 31.8, or roughly **11% of remaining headroom unclaimed**. That would not close a 6.35-point gap, but the LSTM numbers are a floor, not a ceiling.

Half the article is discarded: the 400-token window removes 49% of tokens and truncates 82% of articles, identically for both systems, so the comparison is unbiased, but neither is evaluated on full-document summarization.

ROUGE has known limits: it measures n-gram overlap against a single reference, rewards extractive copying (why Lead-3 scores 39.89), and cannot distinguish a factually wrong summary from a correct paraphrase. Our unsupported-content diagnostic is a lexical proxy, not a factuality judgement, and no human evaluation was performed.

The dataset limits what we can claim: English-only US and UK news from a narrow period, whose summaries are editor-written highlight bullets, so conclusions do not transfer to other languages, genres or summary styles. It is Apache-2.0, and the articles remain the publishers' property and are not redistributed. We also cannot verify what was in the LLM's pretraining data, so its numbers should be read as an upper bound on genuinely held-out performance; our few-shot exemplars come from train, which controls the leakage we can control. On compute, training took 8.73 GPU-hours across all four runs (3.05 for the model itself) plus 4.07 of generation: small in absolute terms, but the LLM's pretraining cost is amortized across all users and invisible here.

7. Conclusion

A 15.3M-parameter LSTM with attention, trained from scratch for 3.05 GPU-hours on 80k pairs, reaches **35.00 ROUGE-1**. A 4-bit Llama 3.1 8B, given the same 400-word window and no training on this dataset, reaches **41.35**. The gap is real ($p < 0.0001$), and it is 6.35 points rather than an order of magnitude.

Three results complicate the expected narrative. Prompt phrasing accounts for 45% of the gap, so much of what looks like capability is specification. Lead-3 outscores four of the five LLM configurations, which says more about ROUGE than either system. And the recurrent bottleneck is not the binding constraint: removing attention costs 14 ROUGE-1, but a 4x smaller encoder window costs nothing measurable.

The engineering conclusion stands regardless. The LSTM is 12x faster and 74x smaller, runs offline, and has bounded failure modes fixable at decoding time. For a task with training data, a latency budget or a data-residency constraint it remains defensible. For a task with no training data it has no answer at all, and that, rather than six ROUGE points, is what pretraining bought.

8. References

- Hermann, K. M., Kočiský, T., Grefenstette, E., Espeholt, L., Kay, W., Suleyman, M., & Blunsom, P. (2015). Teaching Machines to Read and Comprehend. *NeurIPS*.
 - See, A., Liu, P. J., & Manning, C. D. (2017). Get To The Point: Summarization with Pointer-Generator Networks. *ACL*.
 - Bahdanau, D., Cho, K., & Bengio, Y. (2015). Neural Machine Translation by Jointly Learning to Align and Translate. *ICLR*.
 - Luong, M.-T., Pham, H., & Manning, C. D. (2015). Effective Approaches to Attention-based Neural Machine Translation. *EMNLP*.
 - Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to Sequence Learning with Neural Networks. *NeurIPS*.
 - Lin, C.-Y. (2004). ROUGE: A Package for Automatic Evaluation of Summaries. *Text Summarization Branches Out*.
 - Wu, Y., et al. (2016). Google's Neural Machine Translation System. *arXiv:1609.08144*. (GNMT length penalty.)
-

Appendix A: Exact prompts

Reproduced verbatim from `src/llm/prompts.py` and the per-run `.meta.json` files. Both variants are run zero-shot and few-shot ($k = 4$); the few-shot exemplars are passed as real prior conversation turns rather than pasted into a single message, and are drawn from the **training** split only.

Variant A: "plain"

Natural summarization request with no dataset-specific styling.

System prompt

```
You are a helpful assistant that writes concise news summaries.
```

User message template

```
Summarize the following news article.
```

```
Article:  
{article}
```

```
Summary:
```

Variant B: "style-matched"

Specifies CNN/DailyMail highlight length, sentence count, and register.

System prompt

```
You write summaries in the style of CNN/DailyMail article highlights: 3 to 4 short, declarative,
```

User message template

```
Write the highlights for the following news article. Output only the highlight sentences as a sir
```

```
Article: {article}
```

```
Highlights:
```

Few-shot exemplars ($k = 4$)

Identical for both variants (same seed). Articles are truncated to the same 400-word window; only their reference summaries are shown here.

1. grant minchion, 23, had the nunchucks in the glovebox of his ice cream van . they were discovered by police along with a lock knife in a draw with money . stanley knife and throwing knife also found during search, a court was told . he claimed knives were useful for opening stock including chocolate flakes . said that he was given throwing knife by a 'kid' in exchange for a cornetto . pleaded guilty to two counts of possessing offensive weapon and possessing bladed article in public place . no allegation that any of the weapons had been used to commit a crime .
2. 15,000 people are killed trying to cross train tracks every year . a safety panel said almost half of the deaths were in mumbai . it said 'no civilized society can accept such massacre' . it blamed outdated technology and lack of infrastructure .
3. female soldier killed, four other troops wounded in roadside bombing . karine blais, 21, is second canadian female soldier killed in afghanistan . there have been 117 canadian troop deaths in the afghan war .

4. small toy expands to 400 times its size, posing an ingestion risk, consumer commission says . 8-month-old baby from texas needed surgery after ingesting marble-size water balz . consumer product safety commission urges consumers to stop using product . also recalled are growing skulls, h2o orbs and fabulous flowers .

Generation settings. Greedy decoding (temperature 0), `max_tokens = 200`, batch size 8. Input condition `truncated_400_words` for the four headline settings; the unmatched control uses the untruncated article.

Appendix B: Contribution statement

The project decomposes into five workstreams of comparable weight.

Group CP-468-D, AI, 17.

Member	Workstream	Specific responsibilities
Mohanad Bahammam	Data & preprocessing	Dataset acquisition and licensing check; frozen seeded splits and the shared head-to-head subset; word-level tokenizer, normalization, and abbreviation-aware sentence splitting; train-only vocabulary construction and coverage analysis; leakage controls. Files: <code>src/data/*</code> .
Yakup Bastug	Model implementation	BiLSTM encoder and the bridge to the decoder state; Bahdanau, Luong, and no-attention modules with source masking; LSTM decoder with input feeding; weight tying; greedy and beam search with length penalty and trigram blocking. Files: <code>src/models/*</code> .
Orhan Gundogan	Training & performance	Training loop, label smoothing, bucketed batching, early stopping and LR scheduling; the four experiment configs; the memory and MPS-shape performance work described in §2.1. Files: <code>src/train.py</code> , <code>configs/*</code> , <code>scripts/train_all.sh</code> .
Ayuub Hagi	LLM baseline	Prompt variant design and the zero-/few-shot protocol; the local MLX backend and the API backend; input-parity design; token, latency, and compute accounting; resumability. Files: <code>src/llm/*</code> .
Khaled Mobarak	Evaluation, analysis & report	ROUGE harness with bootstrap CIs; Lead-3 validation against the literature; length and abtractiveness bucketing; repetition/OOV/unsupported-content diagnostics; qualitative selection and error categorization; this report. Files: <code>src/evaluate.py</code> , <code>src/qualitative.py</code> , <code>scripts/collect_results.py</code> , <code>reports/*</code> .

Per-member ownership detail, meaning the files each member owns and the results they are accountable for, is in `TEAM.md` in the repository.

Every member should additionally be able to explain the whole pipeline end to end; the demo (Appendix D) exercises all five workstreams in one run.

Appendix C: AI-use disclosure

Tools and use. During this project we used **Claude (Anthropic)** for the following: (1) the initial model, training loop, and evaluation scripts; (2) suggesting evaluation edge cases and diagnostics; (3) report review.

Results. We did **not** use AI to generate our experimental results. Every number reported here was produced by executing the committed code on our own hardware; `scripts/collect_results.py` reads the run artifacts directly and will not emit a figure whose artifact is missing, so all results are reproducible from the repository by anyone.

Responsibility. All AI-assisted code was reviewed and tested by the authors, who ran the full pipeline end to end and are responsible for its correctness. Each author owns a specific component and can account for its design decisions (see Appendix B and `TEAM.md`).

LLM as object of study. Separately from the above, an LLM is the *subject* of the experiment: Llama 3.1 8B Instruct is the baseline system in §2.2. That use is methodological rather than authorial and is documented in full in §2.2 and §4, with the exact prompts in Appendix A.

Appendix D: Repository and demo

- Code: <https://github.com/KhaledM0barak/lstm-vs-llm-summarization>
- Demo video (7:40): [OneDrive](#) (Wilfrid Laurier sign-in required. Runtime 7:40; produced in a single take by `scripts/walkthrough.sh`, which is committed, so the video can be regenerated.)
- Testing and recording runbook: `DEMO.md`

Appendix E: Implementation notes

Three implementation details from §2.1, in full. Each was found by measuring rather than by reasoning about the code, and two of them were the difference between a run that finishes and one that does not.

1. Attention masking. Padding is masked before the attention softmax. Without the mask the decoder places probability mass on `<pad>` for every short article batched with long ones, silently corrupting the context vector rather than raising an error.

2. Chunked vocabulary projection. Projecting decoder states to the 50k vocabulary in a single `matmul` materializes a `(B, T, V)` tensor of 1.28 GB in `fp32` at batch 64 and 100 steps, which a mask-indexed loss then copies twice more. Peak memory drove the 24 GB machine 9.5 GB into swap and degraded throughput from 1.1 s/batch to 3.2 s/batch and worsening. Applying the projection in 16-step chunks with `F.cross_entropy` (native label smoothing and `ignore_index`, so no masked copies are materialized) holds peak RSS at 1.15 GB.

3. Padded-shape quantization. The dominant cost, and the least obvious. PyTorch's Metal (MPS) backend compiles a kernel per distinct tensor shape, and length-bucketed batching produces a near-unique `(batch, src_len, tgt_len)` triple almost every step. Training was therefore spending most of its wall-clock in shader compilation, with `MTLCompilerService` pinning two cores, rather than in arithmetic. Rounding padded lengths up to multiples of 64 (source) and 16 (target), and dropping each pool's ragged final batch to fix the batch dimension, collapses thousands of shapes into a few dozen that compile once and are reused:

Configuration	Before	After	Speedup
Bahdanau attention + input feeding	2537 s	73.7 s	34x
Multiplicative attention, batched	2695 s	49.1 s	55x

(one epoch over 3,840 examples, batch 64, identical hardware)

The cost is under 2% of training examples per epoch and a little extra padding. This is an accelerator-specific detail rather than anything about summarization, but it is the kind of finding that only appears if throughput is measured rather than assumed, and without it this project's four training runs would have taken an estimated 20+ hours instead of 8.7.

A related failure mode, found while building the demo. The Metal backend does not raise when a command buffer runs out of GPU memory; it returns whatever was in the buffer. With a local LLM resident on the GPU, the LSTM silently produced empty or degenerate the the the a the output, which is indistinguishable from a broken model. `src/demo.py` therefore decodes a known article at startup, checks the output's unique-token ratio, and falls back to CPU with a warning if it looks degenerate.

Appendix F: Full result tables

Generated by `python scripts/collect_results.py`, which reads the run artifacts directly; see also `results/results.md`.

F.1 All systems (ROUGE F1 x100, 95% bootstrap CI, n = 500)

System	ROUGE-1	ROUGE-2	ROUGE-Lsum	Len
LLM B, zero-shot	41.35 [40.44, 42.20]	18.07 [17.25, 18.84]	38.14	82.1
LLM B, zero-shot, full article	40.87 [39.95, 41.70]	17.70 [16.92, 18.50]	37.65	84.0
LLM B, few-shot k=4	40.48 [39.62, 41.34]	16.58 [15.82, 17.36]	37.58	79.1
Lead-3 baseline	39.89 [38.87, 40.93]	17.60 [16.58, 18.60]	36.35	86.2
LLM A, few-shot k=4	39.40 [38.60, 40.21]	14.90 [14.23, 15.58]	36.15	81.1
LLM A, zero-shot	38.52 [37.63, 39.34]	14.91 [14.28, 15.56]	34.65	109.9
LSTM + attention (beam 4)	35.00 [34.03, 36.04]	13.75 [12.85, 14.64]	32.25	48.2
100-token window	34.02 [33.10, 34.91]	13.44 [12.59, 14.29]	31.49	41.8
unidirectional	33.12 [32.21, 34.08]	12.64 [11.84, 13.49]	30.37	45.8
LSTM (greedy)	32.60 [31.66, 33.62]	12.05 [11.25, 12.85]	30.57	48.3
LSTM (no trigram block)	29.65 [28.69, 30.71]	10.92 [10.14, 11.76]	26.99	51.8
no attention	20.97 [20.32, 21.67]	3.47 [3.15, 3.81]	19.40	38.2

F.2 Paired bootstrap vs. LSTM + attention (10,000 replicates)

System	Δ ROUGE-1 [95% CI]	p	Δ ROUGE-2	p	W/L
LLM B, zero-shot	+6.35 [+5.34, +7.39]	<0.0001	+4.33	<0.0001	358/142
LLM B, few-shot	+5.48 [+4.43, +6.54]	<0.0001	+2.83	<0.0001	348/152
Lead-3	+4.90 [+3.85, +5.94]	<0.0001	+3.86	<0.0001	333/167
LLM A, few-shot	+4.40 [+3.42, +5.41]	<0.0001	+1.16	0.014	329/171
LLM A, zero-shot	+3.52 [+2.52, +4.55]	<0.0001	+1.17	0.012	314/185
100-token window	-0.98 [-1.96, +0.02]	0.053 (<i>n.s.</i>)	-0.31	0.495 (<i>n.s.</i>)	235/265
unidirectional	-1.87 [-2.74, -0.98]	0.0001	-1.11	0.003	210/284
LSTM (greedy)	-2.40 [-3.23, -1.53]	<0.0001	-1.69	<0.0001	204/291
LSTM (no trigram block)	-5.35 [-6.03, -4.67]	<0.0001	-2.83	<0.0001	62/330
no attention	-14.02 [-15.02, -13.00]	<0.0001	-10.28	<0.0001	56/442

F.2b Input-parity check: full article vs. matched 400-word window

Paired bootstrap, variant B zero-shot, n = 500.

Metric	Δ (full - matched)	95% CI	p	W/L
ROUGE-1	-0.47	[-0.91, -0.03]	0.034	177/195
ROUGE-2	-0.37	[-0.81, +0.07]	0.099 (<i>n.s.</i>)	164/201
ROUGE-Lsum	-0.49	[-0.93, -0.05]	0.028	172/198

F3 Behavioral diagnostics (means)

System	Dup-trigram	Novel-bigram	Unsupported	OOV	Empty
LSTM + attention	0.000	0.080	0.015	0.000	0.000
LSTM (greedy)	0.000	0.218	0.065	0.000	0.000
LSTM (no trigram block)	0.282	0.061	0.008	0.000	0.000
no attention	0.000	0.704	0.530	0.000	0.000
unidirectional	0.000	0.088	0.024	0.000	0.000
100-token window	0.000	0.150	0.040	0.000	0.000
LLM A, zero-shot	0.007	0.521	0.206	0.021	0.000
LLM A, few-shot	0.005	0.500	0.177	0.021	0.000
LLM B, zero-shot	0.009	0.246	0.055	0.029	0.000
LLM B, few-shot	0.010	0.370	0.100	0.024	0.000
Lead-3 baseline	0.011	0.000	0.000	0.023	0.000

OOV is 0.000 for every LSTM system **by construction**, because `<unk>` is suppressed at generation, so the model cannot emit an out-of-vocabulary token. See §5.

F4 ROUGE-1 by source length and by reference abtractiveness

System	Short (≤ 534)	Medium (534-834)	Long (> 834)
LLM B, zero-shot	42.87	41.29	39.89
Lead-3	42.00	40.81	36.87
LSTM + attention	35.69	36.16	33.15
100-token window	35.57	35.37	31.14
no attention	21.71	20.40	20.81

System	Extractive	Mixed	Abstractive
LLM B, zero-shot	46.29	42.09	35.67
Lead-3	45.35	39.72	34.61
LSTM + attention	40.04	34.44	30.52
no attention	22.15	21.46	19.32

F5 Training and inference cost

Run	Parameters	Epochs	GPU-hours	Best val loss	Val PPL
base	15,347,280	5	3.05	4.6219	35.6
no_attention	15,150,416	5	2.67	5.6369	121.7
unidirectional	14,558,800	5	1.90	4.7204	40.0
short_context	15,347,280	5	1.11	5.1179	65.5

Total training **8.73 GPU-hours**. LSTM inference: 500 summaries in 115.6 s = **0.231 s each** (259/min), model 61 MB in fp32.

LLM setting	Input tok	Output tok	Wall-clock	Throughput
A zero-shot	270,587	62,501	25.2 min	19.9 / min
B zero-shot	318,087	48,387	23.7 min	21.1 / min
A few-shot k=4	1,234,587	48,433	76.5 min	6.5 / min
B few-shot k=4	1,328,087	47,914	76.5 min	6.5 / min
B zero-shot, full article	500,578	49,471	42.0 min	11.9 / min

Total LLM generation **4.07 GPU-hours** for 2,500 summaries, monetary cost **\$0.00** (run locally). Note the full-article condition costs 77% more generation time than the matched window for a *lower* score (§6.3).

F.6 Per-epoch validation curves

Validation perplexity after each epoch. Every run hit the 5-epoch cap with validation loss still improving, so early stopping never fired and none of these models is converged.

Run	Ep 1	Ep 2	Ep 3	Ep 4	Ep 5
base	335.8	70.0	47.0	39.4	35.6
unidirectional	305.9	73.8	52.7	44.9	40.0
short_context	320.2	117.8	85.2	72.8	65.5
no_attention	335.4	208.9	159.5	135.4	121.7

The per-epoch gain on the base run halves consistently (265.8, 23.1, 7.6, 3.8), so a geometric extrapolation puts the converged value near 31.8, about 11% below the value we report. That is the basis for calling our LSTM numbers a floor in §6.5.

Two things are visible here that the final-epoch column alone hides. The no-attention run improves at the same *rate* as the others but from a far worse position and never recovers, which is consistent with a capacity limit on what it can represent rather than an optimisation failure. And short_context sits between the two throughout, despite scoring within noise of the base model on ROUGE (§4.3): it is a measurably worse language model that produces summaries of indistinguishable quality, which is the clearest single piece of evidence for lead bias in this report.

Appendix G: Error-analysis evidence

The claims in §5 are verified against the data rather than asserted. This appendix reproduces the evidence. Full side-by-side comparisons for all thirteen behaviour-selected examples are in `results/qualitative.md`.

G.1 The OOV failure is real but invisible in the OOV metric

<unk> is suppressed at generation, so every LSTM system reports an OOV rate of exactly 0.000. Lead-3, which copies real article text, shows the true rate for this corpus (0.023). **334 of 500 references (67%) contain at least one token outside the 50,000-type vocabulary**, so the failure necessarily surfaces as substitution or omission instead:

Reference contains	Model produced	Failure
fredric brandt	dr. frederic brandt	In-vocabulary misspelling of a real person , a factual error with no metric signature
sportsmail have teamed up with golfbidder	sportsmail have teamed up with the to offer...	Entity dropped , sentence left ungrammatical

G.2 Unsupported-content is a lexical proxy, not a factuality measure

Constructed cases against a fixed source article ("*the mayor announced on tuesday that the city will build a new hospital in the eastern district at a cost of forty million dollars*"):

Candidate summary	Truth	Unsupported-content
"the city plans to construct a medical facility costing \$40m"	correct	0.857
"the mayor announced the hospital will be built in the western district at a cost of four million dollars"	false	0.300
"the mayor announced on tuesday that the city will build a new hospital"	correct (verbatim)	0.000

The faithful paraphrase scores nearly **three times worse** than the fabrication. The metric counts lexical novelty, which correlates weakly with factual error and here inverts. This is why §5 declines to call the LLM's 0.055 rate a hallucination rate.

G.3 What the attention mask prevents

Measured on one short article (3 real tokens) batched with a longer one, using the trained model's attention module:

WITHOUT mask:	0.132	0.094	0.146		0.108	0.107	0.144	0.146	0.123
WITH mask	:	0.354	0.253	0.393		0.000	0.000	0.000	0.000
		real tokens				padding			

62.8% of attention mass lands on padding without the mask, and the resulting context vector differs by 1.83 (L2). It also makes the model's output depend on *padding content*, meaning on which other articles happen to share its batch, which is a correctness violation, not merely a quality loss. `tests/test_models.py` asserts both properties.

G.4 The two systems fail in opposite directions

Largest LLM advantage on a clean case (ROUGE-1 10.5 → 43.1; test example 112). A football match report. The LSTM's two sentences contain four distinct failures: it says the striker scored a **hat-trick** and then that the forward scored a **brace**, meaning three goals and then two, about the same player; the two sentences are near-duplicates in structure, showing repetition evading the trigram filter by varying one word per slot; it ends "to win a win"; and it never names anyone, describing players by role because *Griffiths*, *Westlake* and *Commons* are unreachable. Its **ROUGE-2 is exactly 0.0**: fluent English sharing not one bigram with the reference.

Reference: darryl westlake fired kilmarnock ahead in the 50th minute . kris commons levelled for the home side eight minutes later . sub leigh griffiths netted three goals in a remarkable 19-minute spell . celtic moved eight points clear ...

LSTM (R1 10.5, R2 0.0): the celtic striker scored a hat-trick to guide his team to a handsome 4-1 victory. the celtic forward scored a brace to guide their team to win a win.

LLM (R1 43.1, R2 18.0): Leigh Griffiths scored a 19-minute hat-trick for Celtic against Kilmarnock. The treble-chasing Celtic came from a goal down to record a 4-1 victory. Darryl Westlake's deflected strike gave Kilmarnock a shock lead early in the second half. Celtic took an eight-point lead at the top of the Premiership with the win.

LSTM beats the LLM (57.1 vs. 26.3), and the mechanism is instructive. The LSTM copies the reference's exact framing; the LLM writes a longer summary that is **entirely true** but reports different facts (follower counts, likes, due date) than the reference chose.

Reference: sarah stage, 30, has documented her changing figure via her instagram page throughout her pregnancy ...

LSTM (R1 57.1, 28 tokens): sarah stage, a 30-year-old underwear model and animal rights activist from los angeles, has documented her changing figure via her instagram page throughout her pregnancy.

LLM (R1 26.3, 85 tokens): Sarah Stage, a 30-year-old underwear model, has shared a photo of her barely-there baby bump 10 days before her due date. The model, who has 1.5 million Instagram followers, posted the picture on Monday...

Neither output is wrong. The LSTM scores twice as high because it happened to select the same facts the editor did, which is what ROUGE measures.

G.5 Two fabrications the diagnostics do not catch

Assigning the error categories in `results/qualitative.md` meant checking each suspected fabrication against the full source article rather than the truncated display. Three were confirmed and one was refuted.

Lexical fabrication, partially visible. On a Barcelona match report (9ee69bc0), the full model produced *"barcelona beat almeria on wednesday after a 1-0 defeat of atletico."* The token *atletico* does not occur in the article, and the source says Barcelona won 4-0. Unsupported content is 0.150 here, so the metric registers something, but it flags the unseen word, not the wrong scoreline. The model also missed the story entirely, summarising the league-table footer instead of the trick shot the reference leads on.

Relational fabrication, wholly invisible. On a funeral report (9c616b0a), the model produced *"isabelle was found in a migrant camp last week. her body was found at st peter's."* Isabelle Hyart is the **living mother** who attended the funeral; the victim is her nine-year-old daughter, and St Peter's is the church, not the discovery site. Unsupported content is **0.000**: every content word (*isabelle*, *body*, *migrant camp*, *st peter's*) appears in the article. Only the relations between them are invented, and no diagnostic we compute can see that.

A suspected hallucination, refuted. The output selected as having the highest unsupported content of any LLM summary (0.230, d40bdad8) survives checking: the view count 374,551, the caption *"boy loses best friend"* and the verb *shriek* all occur verbatim in the source. The score reflects paraphrase, not invention, the same conclusion as the constructed cases in G.2, reached on a real example.

Name substitution, confirmed. On a pageant report (b725b494) the model wrote *"claudia james"* for *claudia alende*; *james* occurs nowhere in the 1,088-word article. This is the OOV failure of G.1 surfacing as a plausible in-vocabulary substitute rather than as an OOV count.

Two consequences. First, our faithfulness proxy has a floor: it detects unseen words, not false propositions built from seen ones. Establishing a real hallucination rate for either system needs human annotation, which we did not perform. Second, the fluent-but-wrong mode is not exclusive to the no-attention ablation as §5 might otherwise be read to imply. Attention reduces it by an order of magnitude without eliminating it.

G.6 Error categories for all thirteen examples

Categories were assigned by reading each output against its **full** source article, not inferred from the diagnostics. Where an output appeared to fabricate something, the source was searched for the token before the category was assigned; three fabrications were confirmed and one suspected case was refuted (G.5). Full side-by-side text is in `results/qualitative.md`.

#	Selected as	LSTM	LLM
1	LSTM's strongest case	Extractive echo, faithful; one truncated clause	Content-selection mismatch, no factual error
2	LSTM's weakest case	Lead-bias miss; every named entity omitted	Content-selection mismatch, over-length
3	Largest LLM advantage	Entity fabrication (<code>claudia james</code>)	Faithful and aligned
4	LSTM beats the LLM	Extractive echo, faithful	Content-selection mismatch, no factual error
5	LSTM beats the LLM	Extractive echo, faithful	Content-selection mismatch, over-length
6	Most repetitive LSTM output	Grammatical breakage, quote misattribution	Content-selection mismatch
7	Highest LSTM OOV rate	Lead-bias miss	Format drift (110 tokens)
8	Most unsupported LLM content	Extractive echo, faithful	Refuted: paraphrase, not invention
9	Longest article	Relational error , no lexical signature	Content-selection mismatch
10	Shortest article	Extractive echo, truncation defect	Faithful; best length match in the set
11	Most abstractive LSTM output	Topic miss + fabrication (<code>atletico</code> , wrong score)	Faithful and aligned
12	Longest LLM output	Topic drift, irrelevant splice	Format drift (118 tokens)
13	Shortest LSTM output	Entity omission, temporal conflation	Faithful and aligned

Counted across the thirteen, the LSTM is faithful but incomplete in 5 cases, misses the story through lead bias in 2, produces something false in 3, and is garbled or off-topic in the remaining 3. The LLM breaks down as 2 cases of format drift, 6 of content-selection mismatch (true statements the editor did not choose), and 5 that are faithful and aligned. No LLM output in the set was found to contain an invented fact, including the one selected as the worst offender.

That asymmetry is the finding. ROUGE penalises the LLM's 6 selection mismatches and does not penalise the LSTM's 3 fabrications at all, because a fabrication assembled from source words scores as well as a faithful copy. Two cross-cutting artefacts reinforce it: `dup3` is 0.000 in every LSTM output, including the one selected as *most repetitive*, and `oov` is 0.000 in every LSTM output, including the one selected as *highest OOV rate*.

Appendix H: Reproducing every number

Every figure in this report is regenerated from committed artifacts by `python scripts/collect_results.py`, which reads the run files directly and lists anything it cannot find under "Not yet available" rather than emitting a value. No number in this report was typed by hand into a table.

From a clean clone (Python 3.10+, any platform):

```
git clone https://github.com/KhaledM0barak/lstm-vs-llm-summarization
cd lstm-vs-llm-summarization
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python -m pytest -q # 178 tests
```

Full pipeline, about 13 GPU-hours end to end:

```
python -m src.data.prepare --out-dir data/processed \
    --train-size 80000 --val-size 3000 --llm-test-size 500 --seed 1234
python -m src.data.build_vocab --max-size 50000 --min-freq 2
bash scripts/train_all.sh                # base + three ablations, 8.73 GPU-h
python -m src.llm.baseline --all --batch-size 8    # 4.07 GPU-h
bash scripts/finish.sh                   # score everything, rebuild all tables
```

To reproduce the pipeline validation in §1 on its own:

```
python -m src.evaluate --test-file data/processed/test.jsonl
```

With no `--system` argument, Lead-3 is the only system scored.

Expected: 40.04 / 17.50 / 36.34 against See et al.'s 40.34 / 17.70 / 36.57.

Seeds are fixed at 1234 throughout. The LLM baseline decodes greedily, so it is deterministic given the same model file. LSTM decoding is deterministic on both CPU and MPS; we verified that example 3 scores 36.9 on either device.

Appendix I: Test suite

178 tests, written against the failure modes we actually hit rather than for coverage. They run in about 70 seconds.

File	Tests	Covers
test_data.py	35	Tokenizer, abbreviation-aware sentence splitting, vocabulary construction, padding and masking in the collate function
test_pipeline_edges.py	40	Resume correctness, usage accounting, truncated-file recovery, LLM replay cache, demo rendering width
test_evaluate.py	31	ROUGE configuration, <code>rougeLsum</code> newline handling, bootstrap CIs, paired bootstrap, bucketing, diagnostics
test_models.py	30	Attention masking, beam search, trigram blocking, weight tying, chunked loss equivalence
test_llm.py	26	Prompt construction, few-shot message structure, backend accounting, input truncation
test_integration.py	16	End-to-end training on a tiny slice, generation, scoring

Three of these encode bugs described in this report, so a regression would fail the suite rather than silently change a number: the attention-mask test asserts that padded positions receive exactly zero attention weight (§2.1, G.3); the sentence-splitter tests cover the three cases that shifted Lead-3 from 40.00 to 40.04 (§1); and the paired-bootstrap test constructs correlated systems where independent confidence intervals overlap but the paired test separates them (§3).